Computergrafik
Vektorgrafik mit Java2D

Arnd Vitzthum

Basiert auf Vorlesung „Medientechnik“

von Prof. Hußmann, LMU München

paint-Methode und Graphics-Objekte

- Jedes **`Component-`**Objekt, das auf dem Bildschirm erscheint, wird durch eine vordefinierte Methode **`paint()`** dargestellt.
- Grundprinzip der Grafik-Ausgabe: Überdefinieren von **`paint()`**
- Zeichenfläche wird vom System bereitgestellt und der **`paint()`**- Methode übergeben:
  - **`Graphics`**-Objekt
  - "Adressat" für alle Zeichenbefehle
  - **`public void paint(Graphics g) { ... }`**
- Historisch gewachsene Details in Java:
  - **`Graphics`**-Objekt war bereits in Java 1.1 vorhanden
  - Ab Java-Version 1.2 wesentlich erweiterte Grafikfunktionen (Java 2D)
    - **`Graphics2D`**-Objekt wird übergeben und kann genutzt werden
    - Typanpassung notwendig: **`Graphics2D g2 = (Graphics2D)g;`**

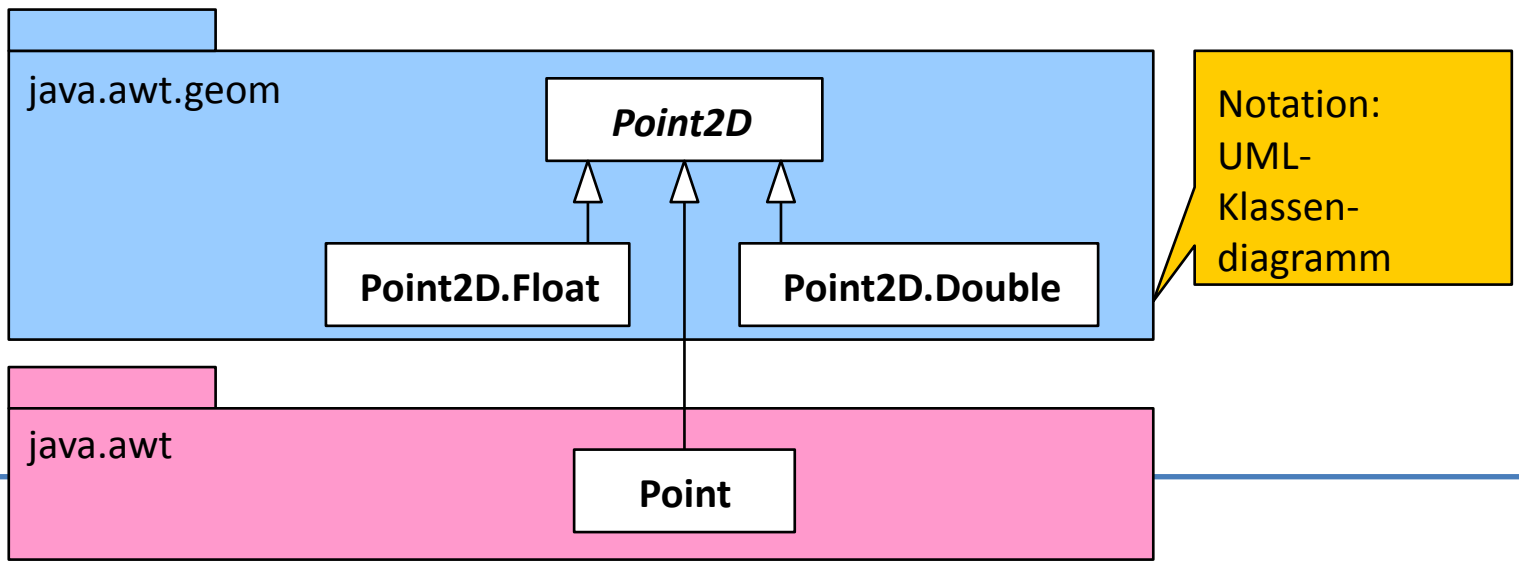
"Hello World" in Grafik-Version

```
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;

public class HelloWorld extends Frame {
    public static void main(String[] args) {new HelloWorld();}
    public HelloWorld() {
        setSize(500, 400);
        setLocation(200, 200);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {System.exit(0);}
        });
        setVisible(true);
    }
    public void paint(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;
        g2.draw(new Rectangle2D.Double(100, 100, 300, 200));
        g2.drawString("Hello World!", 150, 150);
    }
}
```

Punkte

- Ein *Punkt* bezeichnet eine bestimmte (durch x- und y-Koordinaten beschriebene) Stelle der Zeichenfläche.
  - Ein Punkt hat keine Ausdehnung und keine weiteren Attribute
  - Ein Punkt ist verschieden von einem Pixel!
- Java 2D: **`java.awt.geom.Point2D`**
- Muster in Java 2D: Abstrakte Oberklasse mit *inneren Unterklassen*



Pfade

- Ein *Pfad* ist eine Folge von *n* Punkten p_i zusammen mit einer Spezifikation für die Verbindungsart für je zwei Punkte (p_{i-1}, p_i für $i = 2, \ldots, n$)
- Mögliche Verbindungsarten:
  - Keine Verbindung *("move to") – verpflichtend für ersten Punkt*
  - Gerade Verbindung *("line to")*
  - Quadratische Bezier-Kurve (mit 1 Steuerpunkt) *("quad to")*
  - Kubische Bezier-Kurve (mit 2 Steuerpunkten) *("curve to")*
- In Paket **`java.awt.geom:Path2D.Float, Path2D.Double`**

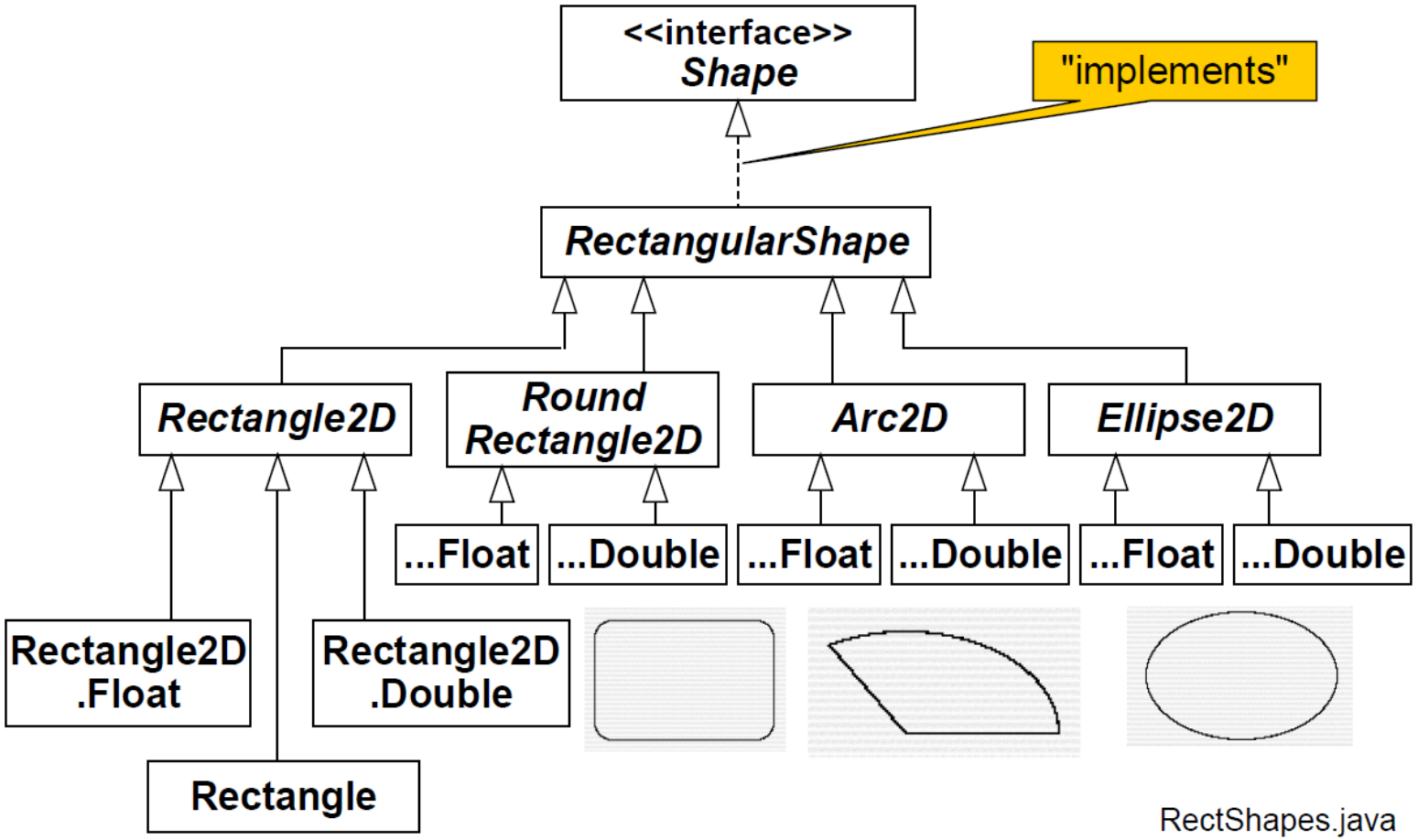
```
public void moveTo(float x, float y)
public void lineTo(float x, float y)
public void quadTo(float xc1, float yc1, float x,float y)
public void curveTo (float xc1, float yc1, float xc2,
        float yc2, float x, float y)
public void closePath() // Linie zum Ende des letzten moveTo
```

Beispiel: Pfad

```
import …

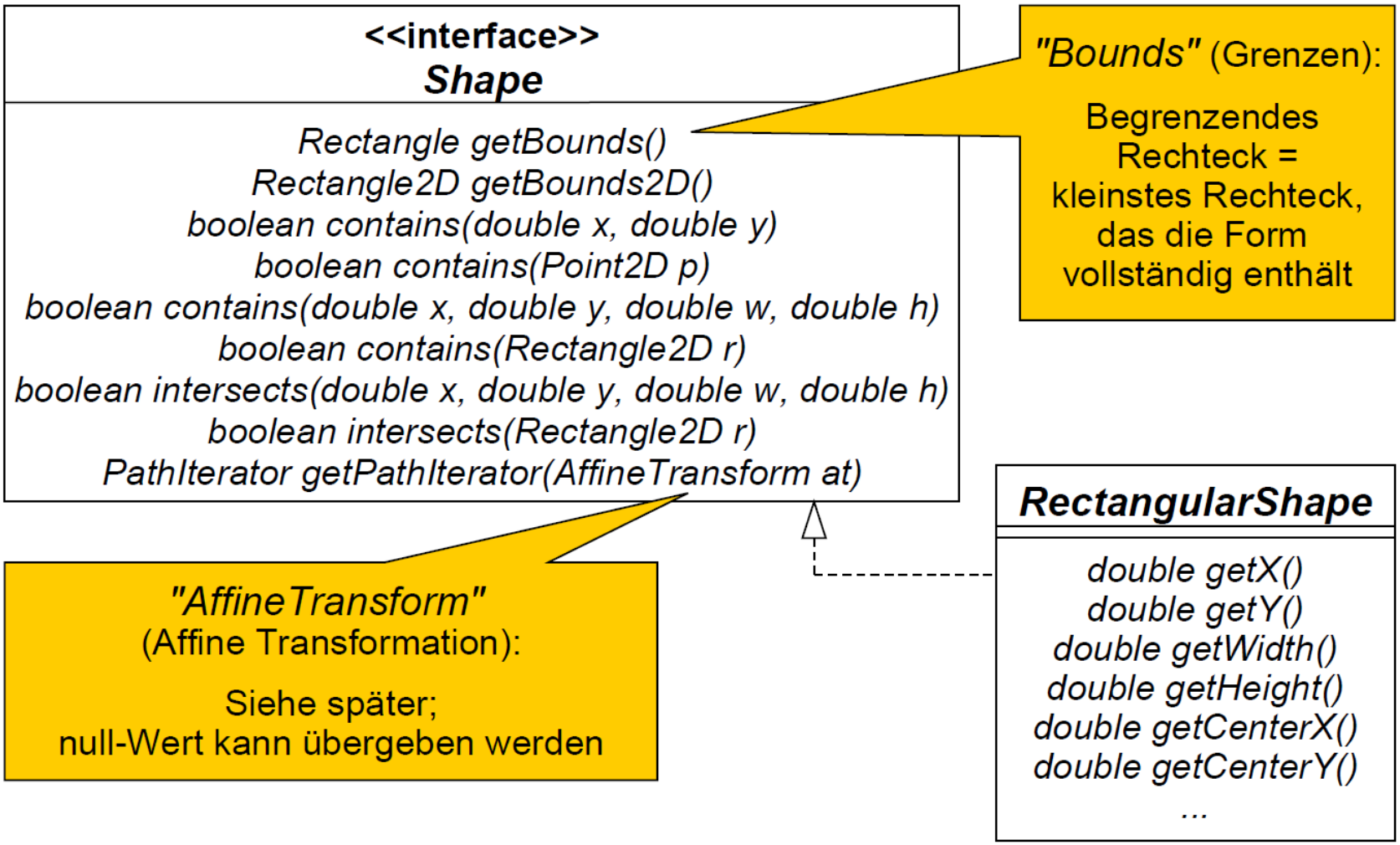
public class Path extends Frame {
public static void main(String[] args) {new Path();}
    public Path() {
         // wie in HelloWorld
    }
    public void paint(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;
        Path2D.Float p = new Path2D.Float();
        p.moveTo(50, 50);
        p.lineTo(70, 44);
        p.curveTo(100, 10, 140, 80, 160, 80);
        p.lineTo(190, 40);
        p.lineTo(200, 56);
        p.quadTo(100, 150, 70, 60);
        p.closePath();
        g2.draw(p);
    }
}
```

Basisformen (1): Flächige Formen

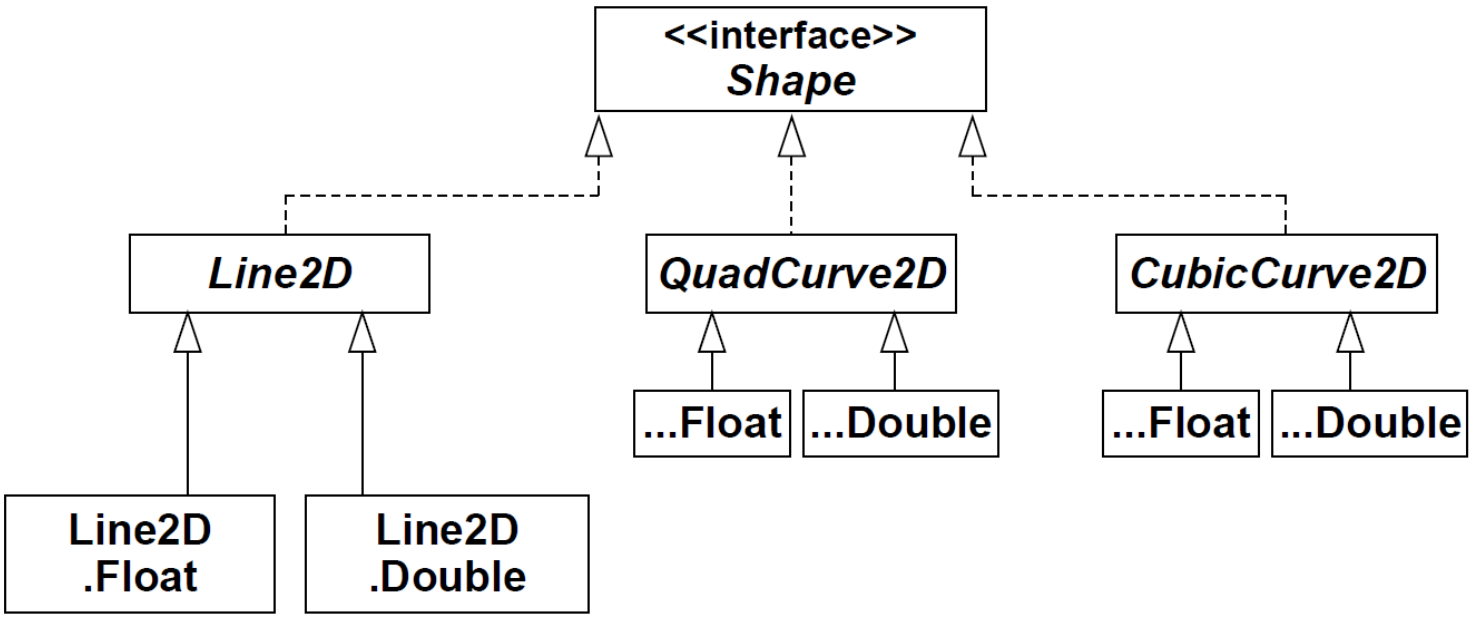


RectShapes.java

Formen

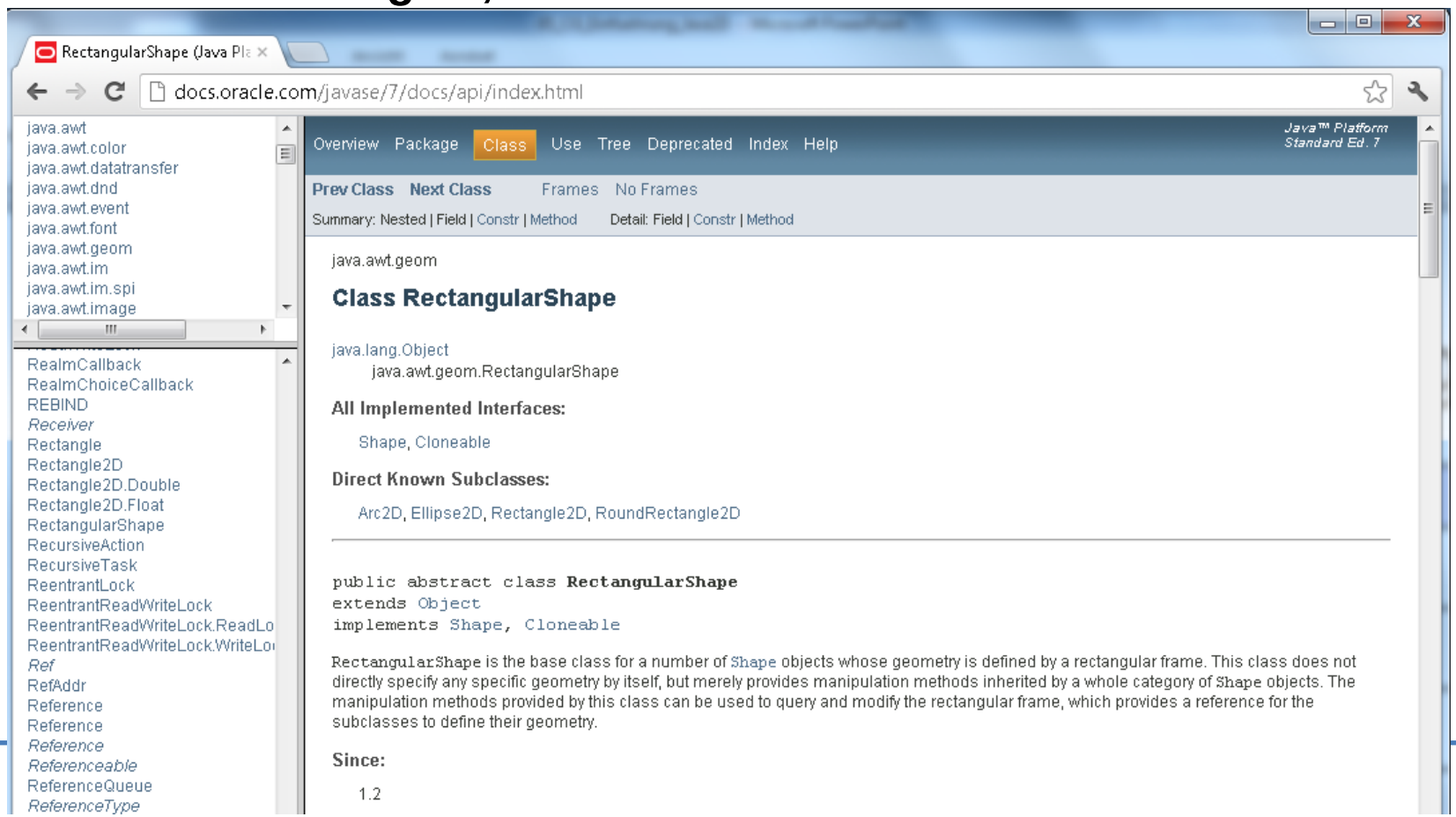

<<interface>>
Shape
Rectangle getBounds()
Rectangle2D getBounds2D()
boolean contains(double x, double y)
boolean contains(Point2D p)
boolean contains(double x, double y, double w, double h)
boolean contains(Rectangle2D r)
boolean intersects(double x, double y, double w, double h)
boolean intersects(Rectangle2D r)
PathIterator getPathIterator(AffineTransform at)
"Bounds" (Grenzen):
Begrenzendes Rechteck = kleinstes Rechteck, das die Form vollständig enthält
"AffineTransform" (Affine Transformation):
Siehe später;
null-Wert kann übergeben werden
RectangularShape
double getX()
double getY()
double getWidth()
double getHeight()
double getCenterX()
double getCenterY()
...

Basisformen (2): Linien und Kurven



Muß ich mir denn das alles merken?

- **NEIN !!! – Wichtig ist, die Dokumentation bedienen zu können**

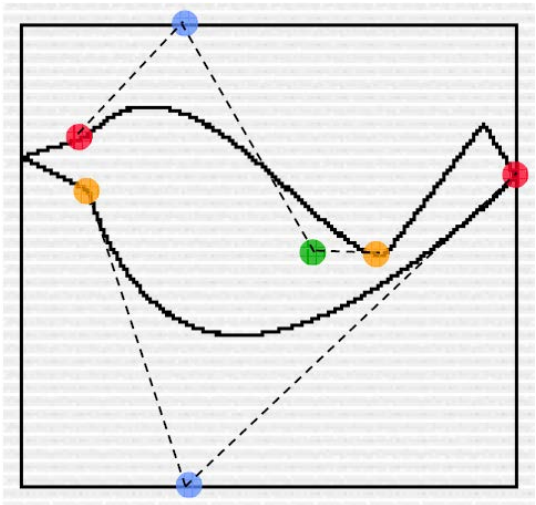


Java-Dokumentation (Auszug)

Modifier and Type	Method and Description
`Object`	**`clone`**`()` Creates a new object of the same class and with the same contents as this object.
`boolean`	**`contains`**`(Point2D p)` Tests if a specified `Point2D` is inside the boundary of the `Shape`, as described by the **definition of insideness**.
`boolean`	**`contains`**`(Rectangle2D r)` Tests if the interior of the `Shape` entirely contains the specified `Rectangle2D`.
`Rectangle`	**`getBounds`**`()` Returns an integer `Rectangle` that completely encloses the `Shape`.
`double`	**`getCenterX`**`()` Returns the X coordinate of the center of the framing rectangle of the `Shape` in `double` precision.
`double`	**`getCenterY`**`()` Returns the Y coordinate of the center of the framing rectangle of the `Shape` in `double` precision.
`Rectangle2D`	**`getFrame`**`()` Returns the framing `Rectangle2D` that defines the overall shape of this object.
`abstract double`	**`getHeight`**`()` Returns the height of the framing rectangle in `double` precision.
`double`	**`getMaxX`**`()` Returns the largest X coordinate of the framing rectangle of the `Shape` in `double` precision.
`double`	**`getMaxY`**`()` Returns the largest Y coordinate of the framing rectangle of the `Shape` in `double` precision.

Beispiel: Ausdehnung (Bounds)

```
Path2D.Float p = new Path2D.Float();
p.moveTo(50, 100);
p.lineTo(70, 94);
p.curveTo(100, 60, 140, 130, 160, 130);
p.lineTo(190, 90);
p.lineTo(200, 106);
p.quadTo(100, 200, 70, 110);
p.closePath();
g2.draw(p);
Rectangle2D r = p.getBounds2D();
g2.draw(r);
```



Konturen und Füllung

- Zwei Möglichkeiten, eine Form anzuzeigen:
  - **`draw()`** zeichnet die Konturen der Form
  - **`fill()`** füllt die Form aus
- Rendering-Parameter, z.B. **`paint`** bestimmen Details, wie Farbe, Strichstärke, Füllmuster
  - Werden beim **`Graphics2D`**-Objekt gesetzt! (nicht beim gezeichneten Objekt)
  - Setzen der Füllattribute: **`setPaint(…)`**
  - Standard-Farben: **`Color.red, Color.blue, ...`**
- Beispiel:

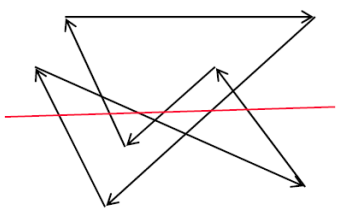
```
Path2D.Float p = new Path2D.Float();
p.moveTo(50, 50); p.lineTo(70, 44); ... ;
p.closePath();
g2.setPaint(Color.red);
g2.fill(p);
```

Innen und Außen (Winding Rules)

- Lege eine Schnittlinie durch die Figur und bestimme damit innen/außen für alle Punkte dieser Linie: Gedanklich der Linie durch die Figur folgen!
- EVEN_ODD-Regel:

  Beginnend bei 0 außerhalb der Figur, erhöhe um 1, wenn eine Kante der Figur überquert wird. Innen: Zähler ungerade, außen: Zähler gerade
- NON_ZERO-Regel:

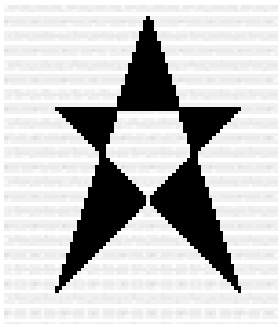
  Beginnend bei 0 außerhalb der Figur, erhöhe (+1), wenn eine (von der Schnittlinie aus gesehen) von links nach rechts laufende Kante überquert wird, erniedrige (–1), wenn eine von rechts nach links laufende Kante überquert wird. Innen: Zähler ungleich null, außen: Zähler gleich null
- Java 2D: Winding Rule ist Parameter bei Konstruktor von **`Path2D.Float/Path2D.Double`**
  - (Default: NON_ZERO)



Beispiel: Winding Rules

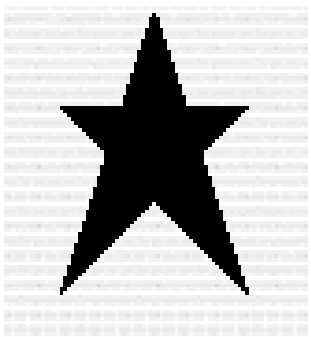
```
public void paint(Graphics g) {
    Graphics2D g2 = (Graphics2D)g;

    Path2D.Float p =
      new Path2D.Float(GeneralPath.WIND_EVEN_ODD);
    p.moveTo(50, 50);
    p.lineTo(100, 50);
    p.lineTo(50, 100);
    p.lineTo(75, 25);
    p.lineTo(100, 100);
    p.closePath();
    g2.fill(p);
}
```



Alternativ:

```
    Path2D.Float p =
      new Path2D.Float(GeneralPath.WIND_NON_ZERO);
    ...
    g2.fill(p);
```



Formen kombinieren (Constructive Area Geometry)

- Ein *Gebiet (area)* der Zeichenfläche kann aus Formen und anderen Gebieten berechnet werden.
- Die wichtigsten Operationen auf Gebieten (gemäss Java 2D):
  - Klasse **`java.awt.geom.Area (implements Shape)`**
  - Konstruktor **`Area(Shape s)`**
  - Hinzufügen (Vereinigung): **`void add(Area rhs)`**
  - Wegnehmen (Differenz): **`void subtract(Area rhs)`**
  - Schneiden: **`void intersect(Area rhs)`**
  - Exklusives Oder: **`void exclusiveOr(Area rhs)`**

